

High-Level System Design

Version: 1.0

Document Control

Version	Date	Author	Description
1.0	2026-05-12	Hardik Patel	Initial Version

Reviewers

Name	Role	Approval Date
Hardik Patel	CTO	2026-05-13

Approvers

Name	Role	Approval Date
Hardik Patel	CTO	2026-05-14

System Overview

Provide a high-level summary of the system.

Include:

- Business purpose
- Core capabilities
- Major users
- Expected scale

Example:

The system enables customers to browse products, place orders, manage payments, and track deliveries through a scalable e-commerce platform.

Business Context

Describe the business problem being solved.

Example:

The organization requires a scalable e-commerce platform capable of supporting increasing online traffic, seamless checkout experience, and reliable order management.

Architecture Goals

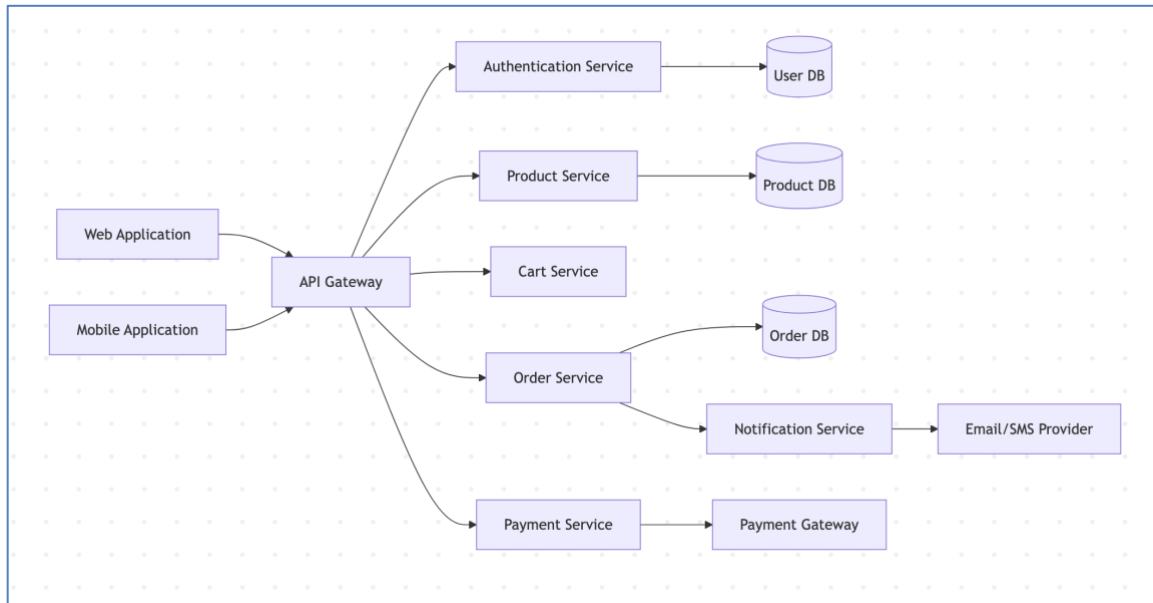
Define primary architectural objectives.

Goal ID	Goal
AG-001	High scalability
AG-002	High availability
AG-003	Secure payment processing
AG-004	Modular architecture
AG-005	Fault tolerance

High-Level Architecture Diagram

Add a high-level architecture diagram.

Example Diagram:



Major System Components

Describe major services/modules.

Component	Responsibility
API Gateway	Central request routing
Authentication Service	User authentication and authorization
Product Service	Product catalog management
Cart Service	Shopping cart operations
Order Service	Order lifecycle management
Payment Service	Payment processing
Notification Service	Email and SMS notifications

Client Applications

Define client applications interacting with the system.

Client	Purpose
Web Application	Customer-facing web platform
Mobile Application	Mobile shopping experience
Admin Portal	Internal management dashboard

External Integrations

List third-party systems integrated with the platform.

Integration	Purpose
Payment Gateway	Payment processing
Logistics Provider	Shipment tracking
Email Service	Notifications
SMS Provider	OTP and alerts

Communication Patterns

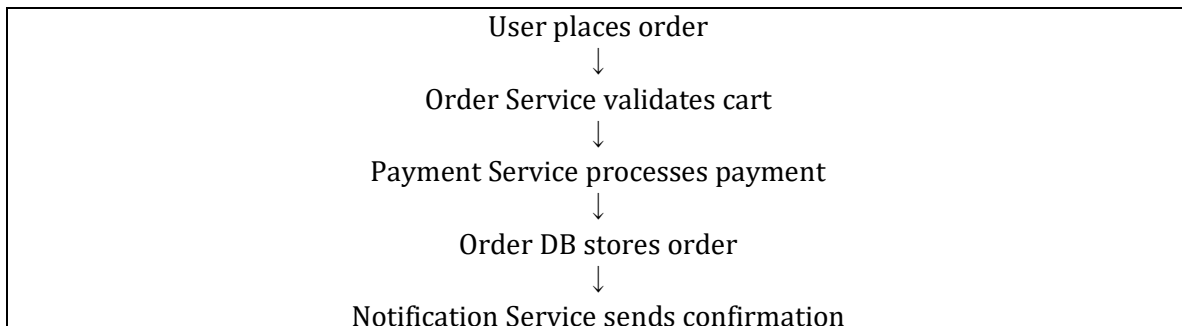
Define how components communicate.

Communication Type	Example
REST APIs	Client to backend communication
Async Messaging	Notification processing
Database Queries	Service-to-database interactions

Data Flow Overview

Describe high-level data movement.

Example Flow:



Authentication & Authorization

Describe high-level security controls.

Area	Design
Authentication	OAuth/JWT-based authentication
Authorization	Role-based access control (RBAC)
MFA	Optional multi-factor authentication
API Security	Token validation and rate limiting

Scalability Considerations

Describe scaling strategies.

Area	Strategy
Web Traffic	Load balancing
Product Catalog	CDN caching
Databases	Read replicas
Services	Horizontal scaling

Reliability & Availability

Describe reliability goals and fault tolerance strategies.

Area	Strategy
Service Failures	Retry mechanisms
High Availability	Multi-zone deployment
Monitoring	Centralized logging and alerts
Backups	Automated backups

Deployment Overview

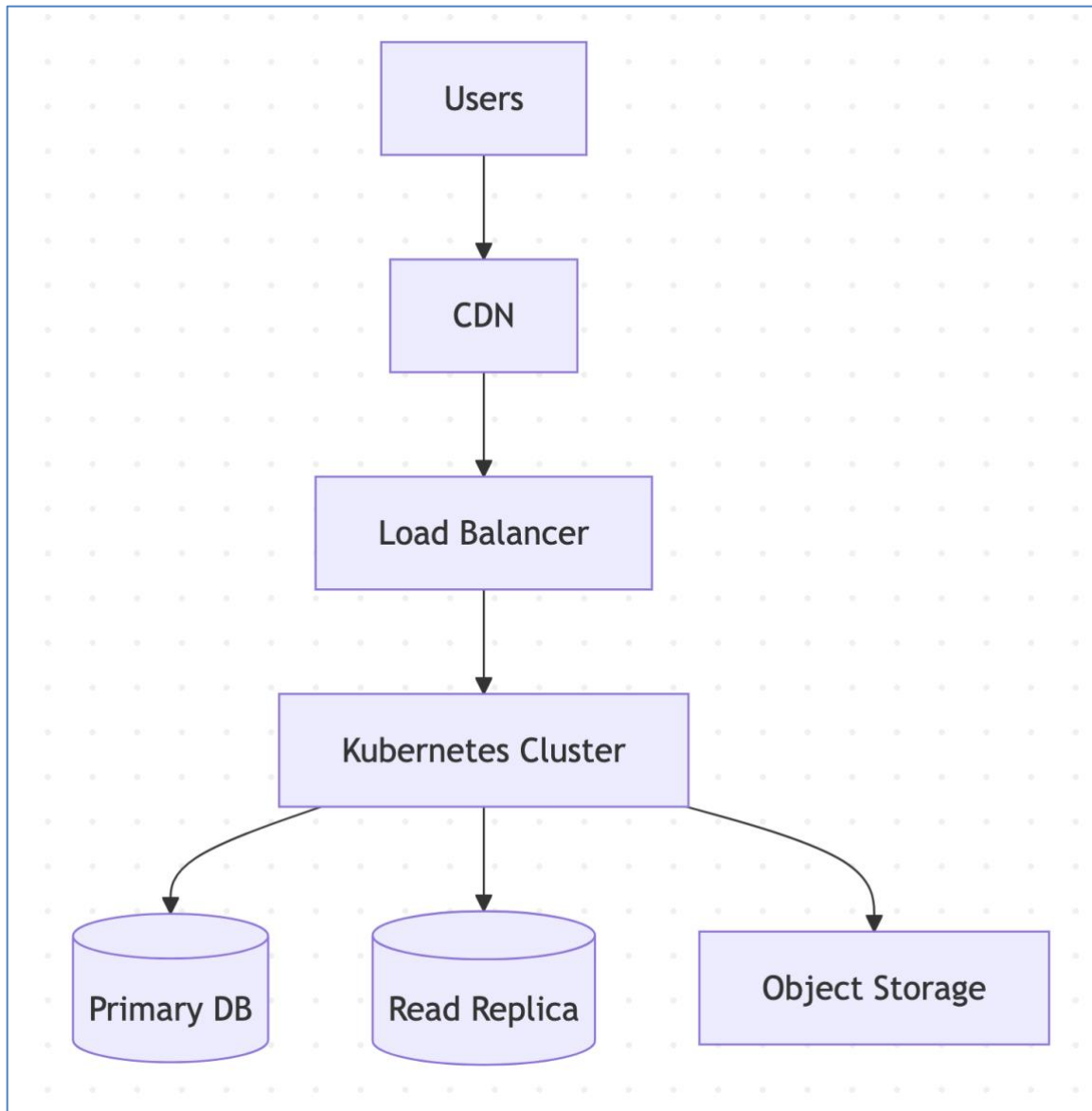
Describe deployment environment at a high level.

Layer	Example
Frontend Hosting	CDN / Static Hosting
Backend Hosting	Kubernetes / Virtual Machines
Database	Managed PostgreSQL
Object Storage	Cloud Blob Storage

Infrastructure Diagram

Add infrastructure-level deployment diagram.

Example:



Security Considerations

Document important security expectations.

Area	Requirement
Encryption	Data encrypted in transit and at rest
Secrets Management	Secure credential storage
Access Control	Least privilege principle

Logging	Security audit logs enabled
---------	-----------------------------

Assumptions

Document architecture assumptions.

Example:

- Cloud provider supports auto-scaling
- Payment provider SLA meets business needs
- Mobile and web clients use same APIs

Constraints

Document known limitations.

Constraint Type	Description
Budget	Limited infrastructure budget
Timeline	MVP delivery within 6 months
Compliance	Regulatory restrictions apply

Risks

Identify architectural risks.

Risk ID	Description	Impact
RISK-001	Database bottleneck during sales events	High
RISK-002	Third-party payment gateway downtime	High
RISK-003	Improper caching strategy	Medium

Future Enhancements

Document possible future architectural improvements.

Enhancement	Purpose
Event-Driven Architecture	Better scalability
Multi-Region Deployment	Improved global availability
Service Mesh	Improved observability and security

Related Documents

Document	Purpose
BRD	Business requirements
PRD	Product requirements
API Specification	API contracts
DB Schema	Data models

Document	Purpose
BRD	Defines business requirements
PRD	Defines product requirements
Feature Specification	Feature-level behavior
API Specification	API contract definitions
DB Schema	Data model definitions
Low-Level Design (LLD)	Internal component-level design